

2026 首届 openvela AI 硬件开发者大赛 · 作品提交模板

一、需提交材料清单

序号	材料	格式 / 说明	提交要求
1	技术报告	.pdf / .docx, 按第二节模板撰写	必交
2	演示视频	≤5 分钟, mp4/mov 等; 须含功能演示、交互操作、AI 能力展示	必交
3	作品展示照片	硬件实物多角度照片 (前视图 / 后视图 / 侧视图 / 俯视图等)	可选 (涉及硬件实物则需提交)
4	海报	.pdf / .jpg 或 .pptx, 一图讲清作品 (用于线下展示)	可选 (入围决赛 / 线下展示则需提交)
5	答辩 PPT	.pptx	可选 (入围决赛提交)

• 项目源码与 AI Coding 日志均在赛事分配的专属仓库 `contest2026_<编号>_<队伍名>` 内, 无需放入压缩包; 评审将直接 clone 该仓库编译运行验证。

• **提交方式:** 将上表材料打包为一个压缩包, 在大赛官网上传。压缩包命名格式为 `<队伍名称>-<作品名称>-<仓库名称>.zip`。

示例: `不会嵌软-VelaSense-contest2026_042_buhuiqianruan.zip`

二、技术报告模板

说明: 本模板通用于各赛道, 非硬件类作品 (如纯快应用) 可跳过硬件相关小节 (如 3.4 的硬件设计与适配、材料清单中的作品展示照片等)。

1、信息表

项目	内容
作品名称	OpenVela R1 全栈适配与多模态能力验证
队伍名称	队伍 441
团队分工	xxluestc（负责人）：BSP 移植、驱动适配、构建烧录、真机验证与材料整理
选题方向	新硬件平台适配

2、摘要

本项目面向 BK7258 R1 开发套件，完成物理 CPU0、CPU1 与 CPU2 上的 NuttX/openvela 启动、内存隔离、系统节拍、板级 bring-up 和跨核通信，并以双屏、GC2145 摄像头、音频和 Wi-Fi 检验真实数据链。全 OpenVela 实验镜像已验证 CPU0 NSH、AP READY、RPTUN CONNECTED 和 CPU2 SMP PASSED；旧混合固件完成双屏、单帧 JPEG、音频回环、联网及按键电源验证，仅作硬件对照。终版按固定提交整合公开 Apache-2.0 BSP，保留来源，不引入他队产品应用；Wi-Fi、多媒体、功耗和稳定性仍待最终实测。

关键词：openvela；NuttX；BK7258；BSP；多核启动；RPTUN；多媒体

3、正文

3.1 绪论

3.1.1 项目背景与问题定义

R1 是一块集成双屏、摄像头、麦克风、扬声器、Wi-Fi、按键、振动器、PSRAM 和 SD-NAND 的 BK7258 多核开发板。原厂方案能够运行成熟的 Armino/FreeRTOS 应用，但如果只在原厂应用上叠加识别、问答或提醒，就无法回答本项目最关心的问题：BK7258 怎样从复位入口启动 openvela，CPU0 怎样拉起 AP，多核之间如何通过 Mailbox、共享内存和 RPTUN 通信，板级初始化怎样把硬件变成 /dev 设备，音视频和网络数据又怎样在有限内存中流动。

因此，本项目把“完成 R1 的可复现全 OpenVela BSP 适配”作为主线，把图形、摄像头、音频和 Wi-Fi 当作检验 BSP 是否真实可用的载荷，而不是只追求上层功能数量。最终成果应从源码、配置、构建、镜像、Flash、CPU 启动、bring-up 到实机证据形成闭环，并能在失败时指出问题所在层级。

3.1.2 技术难点

三颗物理 CPU 与两个 NuttX 层级容易混淆。 物理 CPU0 负责系统首启和 AP 管理；

物理 CPU1 是 AP NuttX 的逻辑 CPU0；物理 CPU2 是 AP NuttX SMP 的逻辑 CPU1。只看到 AP NSH 不能证明 CPU0 也是 openvela，只看到 /proc/cpuinfo 的 processor: 0 也不能证明 CPU2 已上线。

****启动和内存边界必须同时正确。**** CPU0、AP、bootloader、共享区、PSRAM 和用户数据有固定 Flash/RAM 契约。入口地址、VTOR、MSP、链接脚本或镜像偏移任何一处不一致，都可能表现为无日志、AP 超时或 RPTUN 长期停在 CONNECTING。

****芯片层与板级层必须分开。**** GPIO 寄存器、IRQ、SysTick 属于 BK7258 芯片层；按键、LED、屏幕复位、电源脚和 Camera 引脚属于 R1 板级映射。把板级常量写进通用 SoC 代码会破坏复用，也容易把相近开发板误当成同一硬件。

****跨核外设服务不是普通函数调用。**** Wi-Fi、Flash 等能力需要明确控制器所在 CPU、RPMsg 服务、共享缓冲所有权和错误恢复；-107、-16 等应用错误往往只是前面 AP 或服务未就绪的连锁结果。

****多媒体容易触发资源冲突。**** 两块 160×160 RGB565 帧缓冲共需 102400 字节，一帧 640×480 VYUY 图像需 614400 字节，16 kHz/16 bit/单声道音频每秒需 32000 字节，还要与网络、TLS 和 JPEG 缓冲共享 SRAM/PSRAM。

****公开参考代码不能直接等同于本队成果。**** 相同 SoC、相近板卡或他队实测只能证明候选方案，仍需固定来源提交、核对许可证、剔除产品应用，再在 R1 上完成构建和实机验证。

• 3.1.3 创新点与差异化

把 R1 从“原厂 CP + openvela AP”的混合验证推进为 CPU0 与 AP 均运行

NuttX/openvela、CPU2 进入 AP SMP 的全 OpenVela 路线，并以状态机和实机日志明确证明每一层。

建立三条互不混淆的证据基线：CPU0-only 最小 OpenVela 用于研究启动和芯片层；旧混合固件用于确认 R1 硬件；全 OpenVela 镜像用于最终架构和跨核验证。

将“源码 → 配置 → 构建 → ELF/bin → 合包 → Flash → CPU 启动 → bring-up → 设备节点/服务”做成可追溯流程，所有结论绑定提交、镜像哈希、串口日志和通过标准。自建 bk7258-openvela-porting Skill，把来源审查、R1 差异、构建同步、失败处理和证据门禁固化为可重复流程，减少反复猜接口和无证据修改。

3.2 系统方案设计

• 3.2.1 总体架构与数据流

终版系统计划采用以下拓扑：

BootROM / Beken bootloader

|



物理 CPU0: NuttX/openvela

├─ 复位、时钟、UART、IRQ、SysTick、堆、NSH

├─ AP 生命周期管理与故障状态

└─ AP 生命周期监督、RPTUN 主端与 Wi-Fi VNET 诊断

| Mailbox / doorbell / 共享内存 / RPTUN-RPMsg

▼
AP NuttX/openvela

- └─ 物理 CPU1 = AP 逻辑 CPU0
- └─ 物理 CPU2 = AP 逻辑 CPU1 (SMP)
- └─ Board bring-up
- └─ framebuffer / Camera / Audio / netdev / storage
- └─ vision_badge 服务与 NSH 验证工具

主要数据流分为四条：

- 显示：应用像素或状态 → framebuffer → GC9D01 lower-half → QSPI/SPI → 两块 160×160 屏幕。
- 视觉：GC2145 寄存器配置 → I2C 与 DVP → YUV buffer/DMA → PSRAM 帧缓冲 → JPEG → V4L2 /dev/video0 → 应用服务。
- 音频：麦克风 ADC/DMA → PCM capture → 音频服务；返回音频 → PCM playback/DMA → 功放和扬声器。
- 网络：应用请求 → NuttX 网络栈 → 跨核 Wi-Fi 服务 → Beken PHY/RF；DHCP、DNS、TCP/TLS 按顺序建立。

• 3.2.2 方案论证与选型

方案	优点	局限	本项目用途
---	---	---	---

CPU0-only OpenVela	启动链最短，便于排查 UART、IRQ、SysTick、堆和 NSH	无法直接覆盖原厂多核无线与完整外设链	最小救援和芯片层学习基线，已实机通过
CPU0 Armino + AP OpenVela	原厂 Wi-Fi、多媒体和电源路径成熟，R1 硬件验证快	CPU0 实际仍是 FreeRTOS，不满足全 OpenVela 主线	只作为硬件对照和回退镜像
CPU0 OpenVela + AP OpenVela + AP SMP	系统边界统一，可完整学习启动、跨核、驱动和资源管理	移植量大，必须重做服务和外设回归	终版主线：实验镜像已证明 CPU0/AP/CPU2 启动链可行

R1 集成的屏幕、摄像头、音频和无线资源能够在一块板上覆盖图形、多媒体和网络三类 openvela 能力，因此选作验证平台。端侧负责启动、驱动、采集、缓存、设备控制和离线诊断；云端模型只作为可选上层能力，不承担 BSP 正确性的证明。断网或弱网时，设备仍应保留 NSH、按键、电源、显示状态和本地采集自检，不把网络失败伪装成识别结果。

• 3.2.3 关键模块设计

- ****CPU0 启动模块****：初始化 MSP/VTOR、FPU、时钟、低级串口、IRQ、SysTick 和堆，进入 nx_start()，再由 NSH 和 AP 管理应用提供可观测入口。
- ****AP 管理模块****：用明确状态记录 AP 启动、心跳、错误和 generation；失败时保留第一处错误，不让后续网络错误覆盖根因。
- ****跨核通信模块****：通过 Mailbox/doorbell 触发事件，共享内存承载数据，RPTUN/RPMmsg 管理通道生命周期；状态至少区分 CONNECTING 和 CONNECTED。
- ****SMP 模块****：AP 的逻辑 CPU0 拉起物理 CPU2，验证第二调度 CPU、IPI 和双向回调；用 online 位图和 PASSED 状态，而不是只看启动字符串。
- ****板级 bring-up****：将通用 BK7258 驱动与 R1 的引脚、电源、总线和设备映射组合，

按依赖顺序注册字符设备、块设备、网络和多媒体设备。

- ****应用服务层****: vision_badge 用 camera/audio/vision/feedback 等窄接口隔离底层, 未接通的后端明确返回 -ENOSYS, 避免把接口占位写成已完成功能。

3.3 核心算法与技术原理

3.3.1 启动、调度与系统节拍

复位后, 链接脚本给出的向量表、初始 MSP 和程序入口必须与 Flash 中的实际镜像地址一致。bk7258_start.c 完成早期架构初始化, bk7258_lowputc.c 提供调度器启动前的串口输出, bk7258_irq.c 建立中断, bk7258_timerisr.c 配置系统节拍, 最后进入 NuttX 调度。

SysTick 不只是“一个定时器”。NuttX 的延时、超时和时间片都依赖持续递增的系统 tick。项目早期出现 NSH 已能输入、但 sleep 不返回的现象, 说明 UART 和命令解释器已运行, 而系统节拍链未闭合。修正 SysTick 后, sleep 和定时任务恢复, 这一案例用于验证“先找第一处失效的公共依赖”。

3.3.2 多核启动、SMP 与跨核通信

CPU0 先启动并建立共享区, 再释放 AP。AP 的首核完成运行时重定位、堆和调度初始化后, 与 CPU0 建立 RPTUN; 随后 AP 首核通过启动入口、栈和 IPI 机制拉起物理 CPU2。三类状态含义不同:

- AP READY: AP 核心已经运行并能响应管理侧;
- RPTUN CONNECTED: 两端共享内存和通知通道已经完成握手;
- CPU2 SCHEDULER_ONLINE 与 AP SMP PASSED: AP 第二调度 CPU 已上线, 并完成双向调度/回调验证。

实验日志中三者同时出现且 heartbeat 递增, 说明启动和跨核基本链可工作; 终版仍需在固定源码和最终镜像上重复验证。

3.3.3 内存与资源约束

项目使用链接脚本和构建后检查共同约束 Flash 与 RAM。当前基线分区中, bootloader 位于 0x00000000, CPU0 应用位于 0x00011000, AP 位于 0x00165000; EasyFlash、RF 校准和用户数据分区禁止被局部测试镜像覆盖。AP 运行区与 CPU0/SWAP 边界同样必须检查, 不能只看 bin 文件大小。

大缓冲优先放入 PSRAM, DMA 缓冲还要满足硬件可访问范围、对齐和缓存一致性。两块显示帧缓冲、VGA 图像、JPEG 工作区、音频环形缓冲和 TLS 内存不能各自按峰值无限申请, 终版应以实际 map、free 和重复开关测试决定预算。

3.3.4 openvela 设备模型与多媒体机制

NuttX 通过 upper-half/lower-half 把应用接口与硬件实现分离。板级 bringup 调用注册函数后, 应用经文件描述符和 ioctl 使用 /dev/fb*、/dev/video*、/dev/audio*、/dev/mmc* 等设备, 而不是直接调用板级寄存器函数。这样可把 GC2145、GC9D01 或音频控制器的 R1 差异留在 lower-half 和 board 层, 并让应用服务保持稳定。

旧混合固件已用这套路径证明 R1 硬件链：GC2145 经 DVP/DMA 采集到 PSRAM，再由 JPEG 编码；PCM capture/playback 已完成 3 秒回环；双 framebuffer 可驱动两块圆屏。上述结果只证明硬件和候选驱动路径，终版仍需在全 OpenVela BSP 上重新注册和验证。

3.3.5 AI 能力边界

当前主线没有在 BK7258 上部署端侧大模型，也没有可报告的模型量化、端侧推理帧率或算力占用。旧应用实验曾通过 HTTPS 使用云端模型完成 ASR、文本问答、TTS 和短上下文，但暴露出端到端等待和音频断续问题；声网路线已停止，不能列为终版成果。终版报告只在相关代码、配置和最终实测仍保留时说说明云端接口，鉴权机制只写本地配置/设备令牌原则，不公开任何密钥。

3.4 系统实现

- 3.4.1 软件与固件结构

当前队伍仓按职责分为:

chips/bk7258/ 器驱动	CPU0/AP/CPU2、IRQ、SysTick、RPTUN 与控制
boards/bk7258/common/ boards/bk7258/aidk_ai_toy/ nuttx/ tools/bk7258/ app/bsp_diag/ app/vision_badge/ .agents/skills/bk7258-openvela-porting/ evidence/ logs/	通用启动、bring-up、链接脚本和分区规则 R1 引脚、外设、电源与本地 CP/AP 配置 本 BSP 依赖的 NuttX 覆盖文件 双镜像构建、地址检查、镜像编码和打包工具 AP/RPTUN/SMP 与 Wi-Fi VNET 诊断命令 历史应用服务，BSP 稳定后再按接口接回 移植审查 Skill、审计脚本和规则 脱敏后的本地实机证据 大赛 AI Coding 原始日志

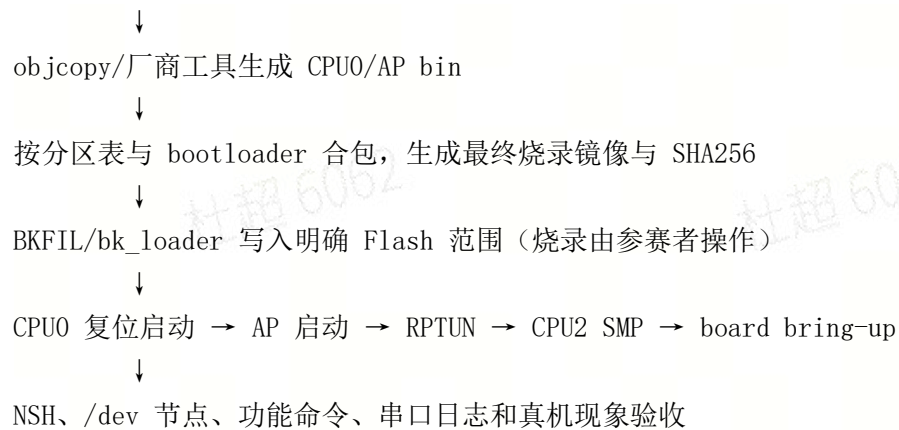
全 OpenVela BSP 主参考固定为 <https://github.com/openvela/contest2026> 135 yongwangzhiqian 的

7079493e73159e00c1b03e60bee6ee69845751cd, 根许可证为 Apache-2.0; Beken SDK 固定为 Embracecactus/bk_avdk_smp@cb080de1655d579c7593ecf504c440997c4c137b。本队只整合 CPU0/AP 启动、SMP、RPTUN、R1 board、Wi-Fi/Flash 服务和构建打包文件, 不复制对方产品应用、UI、云端业务、模型和凭据。文件级边界记录在 SOURCE PROVENANCE.md。

- 3.4.2 从源码到实机的完整流程



map/readelf/nm 检查入口、VMA/LMA、段和边界



烧录地址不是从文件名推测，而是由分区表、链接地址和打包清单共同确定。项目曾用局部写入只更新 CPU0/AP，避开 boot、配对、EasyFlash、RF 校准和用户数据区域；每个候选镜像都应记录输入文件、地址、长度、工具版本、日志和 SHA256，并保留已验证的回退镜像。

• 3.4.3 芯片层与板级适配

- bk7258_start.c: 复位入口与进入 NuttX 前的架构初始化；
- bk7258_lowputc.c、bk7258_serial.c: 早期 UART 和 NuttX 串口设备；
- bk7258_irq.c: 中断控制器初始化与 IRQ 分发；
- bk7258_timerisr.c: 系统节拍；
- bk7258_allocateheap.c: 堆边界；
- bk7258_gpio.c: 与 SoC 相关的 GPIO 操作；
- board/bk7258-r1/src/bk7258_boardinitialize.c: 早期板级初始化；
- board/bk7258-r1/src/bk7258_bringup.c: 驱动注册与文件系统/设备 bring-up；
- bk7258_buttons.c、bk7258_userleds.c: R1 引脚映射与 NuttX board API。

GPIO 实测曾发现初始 LED 引脚映射错误：KEY1 和 KEY3 只能点亮同一颗绿灯。回到 R1 原理图和板级映射修正后，KEY1/KEY2/KEY3 分别产生掩码 0x01/0x02/0x04，并对应红灯、绿灯和双灯。这说明不同 board 的引脚必须留在板级，不能通过猜测修改通用 GPIO 驱动。

• 3.4.4 应用和交互实现

vision_badge 把应用拆为 camera、audio、vision、feedback、workflow 等服务。当前终版仓中的基础版本主要提供接口契约、设备探测和错误传播，未接通的硬件后端返回 -ENOSYS。它的作用是让 BSP 替换时上层调用关系保持稳定，并把“设备不存在”“驱动失败”“云端不可用”分开，而不是用占位代码冒充已完成的多模态产品。

已验证的人机交互还包括：三枚按键和双色 LED；旧混合固件上的双屏动态眼睛；KEY1 长按 3 秒进入产品意义上的关机/深睡，关机后短按和串口回车不会启动，长按 3 秒才接受唤醒。电源行为仍需在终版全 OpenVela 镜像上回归。

• 3.4.5 自建 Skill

自建 Skill 位于 .agents/skills/bk7258-openvela-porting/，不是一段泛化提示词，而是面向本项目的移植流程：

读取来源图，区分上游、厂商、公开候选和本队实测；
检查 R1 原理图/BOM、许可证、固定 commit 和板级差异；
用同步脚本保证队伍仓库板级代码与完整 openvela 工作区一致；
用审计脚本检查项目结构、配置、证据和占位后端；
以“源码存在 → 配置生效 → 构建通过 → 镜像正确 → R1 实机通过”为证据门禁；
对失败记录第一处异常、层级、回退点和下一实验。

仓库现有 Python 回归测试用于检查 Skill 审计规则。最终提交前应把一次真实执行命令和输出放入稳定证据路径。如果最终同时申报“AI 硬件产品创新”，还需新增或明确一个设备运行时 Skill；当前开发 Skill 不应被误写成设备端 Agent Skill。

3.5 系统测试与结果分析（务必提供量化数据）

3.5.1 测试环境

项目	环境
开发板	Agora AIDK AI Device Kit R1，板上丝印版本以最终照片为准
主芯片	BK7258
主机	Windows + Ubuntu 虚拟机；最终报告补充 CPU、内存和虚拟机配置
openvela/NuttX	以终版仓库固定 commit 和最终构建日志为准
Beken SDK	Embracecactus/bk_avdk_smp@cb080de1655d579c7593ecf504c440997c4c137b，来源分支 openvela/v3.1.1.9
编译器/工具	ARM GCC、Beken 打包工具、BKFIL/bk_loader；编译器小版本尚待终版构建日志确认
串口/烧录	COM 端口号仅为当次环境，不作为可移植配置；烧录由参赛者执行

3.5.2 功能测试结果

A. 全 OpenVela 主线

测试项	结果	证据与边界
CPU0 NuttX/openvela 启动	实验镜像通过	R1 进入 NSH, uname -a 显示 NuttX/ARM；终版需保存完整冷启动日志和镜像哈希
AP 启动	实验镜像通过	AP state=READY(2) error=0, heartbeat 在两次查询间递增
RPTUN	实验镜像通过	RPTUN state=CONNECTED(4), epoch 两端为 1
CPU2/AP SMP	实验镜像通过	CPU2 state=SCHEDULER_ONLINE(8), AP SMP state=PASSED(4), online=00000003
全 OpenVela Wi-Fi	未完成	曾出现服务未就绪 -107、忙 -16、link=0；不能用旧混合固件的联网结果代替
全 OpenVela 双屏/Camera/Audio	待终版回归	硬件在旧混合固件已证实，但终版驱动注册、资源归属和重复开关尚需验证

B. CPU0-only OpenVela 基线

测试项	结果	证据
---	---	---
构建与烧录	通过	nutttx.bin 181964 字节，组合镜像 2994176 字节；BKFIL 输出 Writing Flash OK
NSH/UART	通过	复位后进入 NuttShell (NSH)，UART 双向交互
SysTick	通过	修正后 sleep 能按时返回
RAMLOG/dmesg	通过	RAMLOG 注册后 dmesg 能读取启动日志
GPIO	通过	KEY1/2/3 掩码为 0x01/0x02/0x04；红灯、绿灯和双灯行为正确

C. 旧混合固件硬件对照

测试项	结果	实测数据与边界
---	---	---
双屏	通过	/dev/fb0、/dev/fb1 注册，两块 160×160 RGB565 屏完成初始化和整帧写入；动态眼睛方向已修正
GC2145/JPEG	单帧通过	传感器 ID 0x2145；640×480 VYUY 614400 字节，JPEG 22051 字节，567 ms，编码器报告 valid JPEG
JPEG 合成输入	通过	320×240 YU12 115200 字节，输出 4267 字节，105 ms
Camera 中断	单次通过	frame=1、vsync=1、err=0、timeouts=0
音频回环	通过	录制 3 秒得到 96000 字节，peak=517，mean_abs=57，扬声器能正常回放
Wi-Fi/DNS	通过	成功关联、DHCP 获得 IPv4 地址并解析公网域名；不在报告公开 SSID、密码和临时地址
按键电源	通过	KEY1 长按 3 秒关机/深睡；短按和串口回车不启动；长按 3 秒唤醒
云端语音	原型可用但不达标	ASR、问答、TTS 和短上下文曾成功；端到端等待长且流式播放有停顿，不列为终版性能成果

valid JPEG 只证明 JPEG 结构检查通过；该帧尚未完成“导出主机并用独立解码器检查”的证据门禁。单帧数据也不能推出稳定帧率或 100 帧可靠性。

3.5.3 性能和资源结果

指标	条件	结果	结论
---	---	---	---
合成帧 JPEG 编码	旧混合固件，320×240 YU12，单次	105 ms，4267 字节	只作为编码器单次参考
Camera 单帧 JPEG	旧混合固件，640×480 VYUY，单次	567 ms，22051 字节	证明链路，不等同持续帧率
音频数据量	16 kHz/16 bit/mono，实测 3 秒	96000 字节	与理论每秒 32000 字节一致

双屏帧缓冲	160×160 RGB565 × 2	102400 字节	资源预算值，非运行峰值
全 OpenVela CPU/AP 心跳	两次 apctl status	heartbeat 持续增加	证明当次调度存活，不代表长稳
CPU、SRAM、PSRAM 实际峰值	终版镜像	截至报告候选稿生成时尚未完成终版 map、free 和高水位测量	不填写估算值
整机功耗/续航	终版镜像、统一电源与采样工具	截至报告候选稿生成时未测	不用芯片手册典型值替代整机数据
连续运行和冷启动成功率	终版镜像	已有单次成功日志，尚未形成规定次数统计	当前不宣称稳定性百分比

- 3.5.4 可靠性、异常与回退
项目已经记录并复盘多种真实失败：串口被占用导致烧录失败；SysTick 未工作导致 sleep 不返回；RAMLOG 未注册导致 dmesg 不可用；AP/RPTUN 偶发未就绪导致 Wi-Fi 连锁失败；LED 引脚映射错误；Camera/Audio 使用后影响深睡；语音下载间隙导致播放停顿。处理原则是保存第一处异常、定位所属层，再用最小镜像或旧混合固件做 A/B 对照。

终版必须保留三类回退点：已验证的 CPU0-only 镜像、旧混合硬件对照镜像和上一版全 OpenVela 候选镜像。尚未完成的可靠性测试包括多次冷启动、Wi-Fi 断线恢复、Camera/Audio 重复 open/close、长时间运行内存变化和异常断电恢复。

3.6 AI-Native 开发说明

指标	数据
AI Coding 代码占比	待最终分支统计；不以估算值代替可追溯结果
使用的 AI 工具	Codex、MiMo Code；AI 负责检索、起草和审查，硬件结论由实机验证
MCP 工具使用情况	使用本地代码、终端和浏览器工具；未把未验证的 MCP 能力写入成果
Skills 使用与新增情况	自建 bk7258-openvela-porting Skill；用于来源审查、分层、构建与证据门禁
Token 使用总量	官方日志已归集 3 个文件、76 条事件并通过校验；总 Token 无可靠统计时不虚构

3.6.1 AI 工具的实际作用
AI Coding 主要用于四类工作：
从大规模 NuttX、vendor 和 Beken SDK 中检索启动入口、配置项和调用链；
起草板级代码、构建/合包脚本、测试桩和文档，再由人工逐行审查；
对串口日志、ELF/map、错误码和镜像地址做交叉分析，提出下一项最小实验；
把每个节点的源码、配置、构建、镜像和实机证据同步到文档。

AI 不替代实机证据。凡是寄存器、引脚、镜像偏移、硬件状态或性能数据，最终以原理图、上游源码、生成的 .config、ELF/map、构建日志和 R1 串口结果为准。AI 提出的未知接

口若无法在源码中找到定义，不能写入终版。

3.6.2 遇到的问题与解决方式

问题	AI 辅助	人工/实机闭环
CPU 编号和系统归属混乱	对比多份源码和启动日志，整理统一拓扑	用 `uname`、`apctl status`、`SMP online` 位图确认实际系统
公开仓能力容易被误当成本队能力	建立来源等级和证据门禁	固定 `commit`、核对 `LICENSE`，只移植 `BSP` 范围，并在 `R1` 重新验证
烧录脚本多次因路径/PowerShell 参数失败	分析工具命令行和输出条件	以 `BKFIIL/bk_loader` 的 `Writing Flash OK` 和镜像文件为最终判断
Wi-Fi 错误码是连锁结果	从应用错误反向追踪 `AP/RPTUN/服务状态`	先查 `AP READY` 与 `RPTUN CONNECTED`，再执行连接和 `DHCP`
语音听感卡顿	结合分阶段耗时与缓冲日志定位云端等待和下载间隙	结果仍未达标，如实从终版主成果中降级，不继续包装

3.6.3 自建 Skill 与开发约束

`bk7258-openvela-porting Skill` 明确了“不猜接口、不把候选当发布、不跳过构建和实机验证、不整仓覆盖”的规则，并提供同步和审计脚本。它使重复的来源审查、配置检查、能力盘点和证据核对能够按同一标准执行。最终材料应附一次 `Skill` 实际执行记录和测试结果。

`AI Coding` 代码占比和 `Token` 数在当前草案中留空。提交前将以最终分支差异、人工修改记录和平台可导出统计为依据填写，并说明统计范围；无法追溯的数字不写。

3.7 总结与展望

• 3.7.1 成果总结

本项目已经完成 `BK7258 R1` 的 `CPU0-only OpenVela` 最小启动、`UART/NSH`、`SysTick`、`RAMLOG`、按键和 `LED` 实测；全 `OpenVela` 实验镜像已经打通 `CPU0`、`AP`、`RPTUN`、`CPU2 SMP` 的基本启动链；旧混合固件则完成双屏、`GC2145` 单帧/`JPEG`、音频回环、`Wi-Fi/DNS` 和按键电源验证，为终版驱动迁移提供了硬件对照。工程建立了板级/芯片层边界、分区与内存门禁、可回退镜像以及自建移植 `Skill`。

本次终版候选已经把筛选后的公开 `BSP` 代码、许可证来源、本队 `CP/AP` 配置、诊断应用和构建打包工具纳入专属仓。它仍不能宣称“终版全 `OpenVela BSP` 全部通过”，因为当前候选尚需在 `Linux` 工作区完成干净构建，并用同一提交生成的镜像完成 `CPU0/AP/SMP`、`Wi-Fi`、双屏和至少一条多媒体链回归。

• 3.7.2 应用前景与价值

成果首先是一套面向 `BK7258 R1` 的可学习、可复现 `BSP` 与 `bring-up` 样例，可用于多模式终端、陪伴设备、视觉提示器、语音交互终端和教学实验。它的价值不依赖某一个产品外壳：统一的 `openvela` 设备接口和跨核服务可以让上层按需求组合显示、视觉、音频和

联网能力，并降低后续应用验证对原厂封闭应用框架的依赖。

目标用户、年龄、地域、使用偏好和商业模式尚未经过用户调研，不能在本报告中虚构市场规模。终版可将其定位为面向开发者和教学场景的 BK7258 R1 openvela 参考工程；具体消费产品方向留待真实需求验证后决定。

• 3.7.3 不足与后续工作

完成终版候选的 Linux 干净构建，保存生成 .config、ELF/map/bin、构建清单和 SHA256；在终版全 OpenVela 镜像上重新验证 Wi-Fi、双屏、Camera、Audio、深睡和错误恢复；补齐 ELF/map 资源统计、整机功耗、冷启动成功率、连续运行和多次外设开关数据；导出 JPEG 到主机独立解码，并完成规定次数的连续采集，而不是用单帧代替稳定性；在音频底层稳定后再评估实时语音方案，不把高延迟或断续原型当成完成品；继续推进 BLE、OTA、低功耗和安全配置，但每项都沿用源码、配置、构建、镜像和实机证据门禁；补导并校验 logs/，完成密钥扫描、最终 README、真机照片、五分钟内 Demo 视频和官网提交包。

评审维度（分值）	报告对应章节
技术难度（30）	3.2 系统方案设计；3.3 核心算法与技术原理；3.4 系统实现；3.5 系统测试
产品创新性（20）	2、摘要；3.1 绪论（创新点）
项目完整度（20）	3.5 系统测试与结果分析；项目源码；作品展示照片
AI 开发（10）	3.3 核心算法（AI 算法实现）；3.6 AI-Native 开发说明
商业潜力（10）	3.7 总结与展望（应用前景与商业价值）
展示效果（10）	演示视频；海报；答辩 PPT

四、注意事项

- 作品提交截止：**2026 年 9 月 20 日**，请合理安排开发与提交节奏。
- AI Coding 日志由插件自动写入仓内 logs/ 目录，请勿手动删改。
- 作品须为原创，遵循 Apache 2.0 开源协议，不得存在版权、专利及法律纠纷。
- 项目须使用 openvela 系统能力（NuttX 内核仓库除外），且至少落地图形、AI、多

媒体三项核心能力之一。

- 若含 AI 语音唤醒功能，统一使用唤醒词：你好，openvela / Hello, openvela。
- 大赛仅在 GitHub 进行。获奖后需按要求将作品 PR 至 openvela 上游对应仓库的 `dev-ai-contest-2026` 分支。